TEST PLAN

PROJECT ***AUTOMATION EXERCISE***

(E-Commerce Functional Testing)

https://automationexercise.com/

Document Revision History

Date	Version	Description	Author	Reviewer	Approver
18.11.2025.		Test plan created	Ana Markovic		

Introduction **3**
Scope of Testing **4**
In Scope 4
Out of Scope 4
Quality Objectives **5**
Primary Objectives 5
Secondary Objectives 5
Test Approach **5**
Automation Testing 6
Testing Types 6
1. Functional Testing 6
2. UI/UX Testing 6
3. Negative Testing 7
4. Usability Testing 7
5. Compatibility Testing 7
6. Automation Testing 7
Resource and Environment Needs **7**
Test Environment 7
Production Environment (Live Website) 7
Test Data Requirements 8
Tools and Software 8
Access Requirements 9
Constraints 9
Test Cases / End-to-End Scenarios **10**
In-Scope Test Cases 10
TC-01: Navigation menu validation 10
TC-02: Category Accordion Validation 10
TC-03: Brand Accordion Validation 11
TC-04: Features Items / All Products Validation 12
TC-05: Recommended Items Section Carousel Validation 12
TC-06: Search Products via input field 13
TC-07: Browse Products by Category 13
TC-08: Add Product to Cart 14
TC-09: Add Multiple Products to Cart 14
TC-10: Remove Product from Cart 15
TC-11: Checkout Process Validation 15
TC-12: Verify Breadcrumb Navigation 16
TC-13: Contact Us Form Submission 17

TC-14: Product Detail Page Validation 17
TC-15: Product Review Submission 18
Bug reports **19**
Bug list 19
BUG-01: Wrong Products Assigned to Wrong Category 19
BUG-02: Missing 404 Page for Invalid Product Link 20
BUG-03: Misspelled Category Name ("Tshirts") 21
BUG-04: Image Rendering / CSS Issue on Product Page 22
BUG-05: No Limitation on Adding Same Product to Cart 23
BUG-06: Breadcrumb Navigation Not Working 24

Introduction

This Test Plan defines the strategy, objectives, scope, and approach for testing the e-commerce web application available at https://automationexercise.com. Automation Exercise is a publicly available practice website designed for QA and automation testing, which simulates typical e-commerce functionality including product browsing, cart management, checkout flow, and user interactions such as adding/removing products and navigating categories.

The primary goal of this test plan is to verify the **core functional features** of the website from an end-to-end perspective, ensuring that the website behaves as expected under normal and edge-case conditions.

This test plan focuses solely on functional testing of the Automation Exercise website. It does **not cover login** and **registration features**, performance testing, security testing, such as penetration testing, vulnerability scanning, or authentication bypass attempts, which are considered out of scope for this functional verification.

Scope of Testing

This test plan defines the scope of functional testing for the Automation Exercise website. It focuses on validating core e-commerce features such as **browsing** products, **adding** items to the cart, **managing** quantities, etc. The objective is to ensure that end-to-end workflows function correctly from a user perspective.

Testing will be considered complete, or "done," when all functional test cases have passed, all critical and major defects have been resolved, the total amounts in the cart are validated, and breadcrumb navigation works correctly, ensuring a stable and user-friendly end-to-end experience.

In Scope

- Browsing products by categories and subcategories
- Adding products to the cart and verifying quantities
- Calculating and validating total amounts in the cart
- Removing products from the cart
- Payment gateway integration
- Breadcrumb navigation verification
- Notification and message validation related to cart operations

- End-to-end user flows from product selection to cart verification (without login)

Out of Scope

- Login and account creation flows
- Email notifications
- Performance, load, and stress testing
- Security testing

Quality Objectives

Primary Objectives

The primary objective of testing the Automation Exercise web application is to ensure that all existing functionalities operate according to the defined functional and non-functional requirements. Since the platform contains a wide range of UI components, forms, flows, and user interaction scenarios, the main goal is to validate that each feature behaves reliably, consistently, and as expected across different user journeys.

Testing activities will confirm that pages load correctly, elements are visible and interactive, input forms validate data properly, navigation is intuitive, and core flows (such as user registration, login, product browsing, checkout simulation, contact form submission, etc.) function end-to-end without errors.

At the end of the test cycle, the application should demonstrate a stable user experience aligned with documented requirements and acceptance criteria.

Secondary Objectives

Secondary objectives include identifying and reporting all functional defects, UI inconsistencies, and usability issues across the Automation Exercise application.

All discovered issues will be documented clearly in the Bug report section. The testing team will ensure thorough coverage of all main user flows and edge cases so that defects are addressed appropriately before the completion of the release cycle.

Test Approach

The testing approach for the Automation Exercise web application will follow a clear and organized process focused on the **most important** areas of the application and **real user flows**. The overall strategy combines manual functional testing, exploratory and automation testing.

Manual testing will be used to validate navigation, product browsing, adding items to cart, form submissions, user interactions, and visual UI elements. Exploratory testing will address potential edge cases, unexpected behavior, and usability inconsistencies.

Test cases will be designed using positive, negative, and boundary-value techniques, ensuring realistic and varied user inputs. Each defect will be logged with detailed reproduction steps, severity classification, and the impact assessment. [1]

Automation Testing

Automation testing will be integrated alongside manual activities to improve overall test efficiency and ensure long-term product stability. Automated tests will target stable, high-priority, and frequently repeated scenarios.

Automation will be implemented using a modern framework Playwright, supporting fast execution and easy maintenance.

Automation will initially cover the following areas:

- **Smoke Tests** – validation of the core application functionalities
- **API Tests** – verification of key backend endpoints

Criteria for selecting scenarios for automation include frequent repetition, high business impact, functional stability, and increased regression risk.

A complete and up-to-date list of automated test cases will be maintained within the project repository under **/tests/e2e/.**

[1] See section Bug reports for more details

Testing Types

The following testing types will be performed to ensure full coverage of the Automation Exercise application:

1. Functional Testing

Validates that each feature functions according to the defined requirements. This includes product browsing, adding items to the cart, updating quantities, form submissions and navigation flows.

2. UI/UX Testing

Ensures that the user interface is visually consistent, intuitive and aligned with expected design standards. Verifies layout, responsiveness, button behavior, labels, error messages and overall ease of interaction.

3. Negative Testing

Checks how the system behaves when receiving invalid, unexpected or boundary-breaking inputs. This includes invalid form submissions, empty fields, incorrect data formats and improper user actions.

4. Usability Testing

Evaluates how easily a user can complete key actions within the site. This includes clarity of navigation paths, intuitiveness of page structure and the accessibility of essential functions.

5. Automation Testing

Automated test scripts will be executed for high-value, repetitive, and regression-prone scenarios. Automation focuses on improving efficiency, reducing manual effort.

Resource and Environment Needs

This section outlines the required resources, tools and environment necessary to execute the testing activities for the Automation Exercise application. Since all testing will be performed directly on the publicly available live website, environment-related needs are adjusted according to the testing needs.

Test Environment

Production Environment (Live Website)

All testing activities will be executed directly on the official Automation Exercise production site: https://automationexercise.com/

Since no staging or test environment is available, the following considerations apply:

- Only non-destructive test scenarios will be executed
- Automation scripts will be designed to avoid generating unnecessary load or unintended side effects

The production environment is assumed to be stable, available and representative of real user conditions.

Test Data Requirements

Because the environment is live and no isolated test data exists, testing will rely on:

- Generic product data visible on the site (product names, categories, prices)
- Temporary session-based data (cart items, filters, sorting selections)
- Synthetic input data for forms, ensuring no sensitive or destructive data is submitted

Negative test data will be crafted manually.

Tools and Software

Browsers

- Google Chrome (primary browser for E2E testing)
- Mozilla Firefox

Operating Systems

- Local development: Windows 10

Automation Framework

- **Playwright**
  - Used for E2E tests
  - Used for API tests

Programming Language

- JavaScript (Node.js)

Test Reporting

- Playwright HTML Report (generated automatically)
- GitHub Action artifacts used for storing reports & screenshots after each run

Continuous Integration / Continuous Delivery

- **GitHub Actions**
  - Runs API tests followed by E2E tests
  - Retries failed tests once
  - Does **not** block merges when tests fail
  - Uses non-headless mode for E2E debugging
  - Saves screenshots, videos, and reports as artifacts

Version Control System

- **Git**
  - Standard Git branching model
  - **main** used as primary stable branch
- **GitHub**
  - Remote repository hosting
  - CI workflows triggered on merges to main/master

Bug Tracking & Documentation

- **Bug report section** inside test plan or test management docs
- Used to log:
  - Reproduction steps
  - Expected vs actual behavior
  - Test data
  - Screenshots
  - Severity & priority

Access Requirements

- Access to the live website (no authentication required)
- Access to test documentation and repository
- Permissions to run automation scripts locally

Constraints

Since a production-only environment is used:

- Tests must avoid actions that create persistent or unwanted impact
- Automation must be rate-limited to avoid excessive load or spamming the live system

Test Cases / End-to-End Scenarios

This section documents the selected end-to-end test cases for the Automation Exercise application. Each test case includes ID, name, pre-conditions, steps and expected results.

The scenarios cover key user workflows, UI interactions, and functional validations to ensure the application meets its functional requirements and provides a smooth user experience.

TC-01: Navigation menu validation

- **Pre-conditions:** User is on homepage
- **Steps:**
    1. Observe the main navigation menu at the top of the page
    2. Hover over each menu item
    3. Click on each item and verify that it navigates to the correct page

- **Expected Result:**
    - All main menu items are visible and clickable
    - On hover menu item change color to orange
    - Clicking links navigates to the correct page

TC-02: Category Accordion Validation

- **Pre-conditions**: User is on homepage
- **Steps**:
    1. Scroll to the sidebar section showing Categories
    2. Verify that all top- level categories are listed
        a. Woman
        b. Man
        c. Kids

3. Click each category accordion header and verify that the panel expands/collapses correctly.
4. After expanding a category, validate that only products belonging to the selected category are displayed.
    a. Woman
        i. Dress
        ii. Tops
        iii. Saree
    b. Man
        i. T-Shirts
        ii. Jeans
    c. Kids
        i. Dress
        ii. Tops & Shirts
5. Check that the displayed category names and product labels **do not contain spelling errors**.

- **Expected Results:**
    - All categories are visible and correctly labeled
    - Accordion panels expand/collapse as expected
    - Product lists correctly reflect the selected category
    - No spelling issues are present in category or product names

---

TC-03: Brand Accordion Validation

- **Pre-conditions:** User is on the homepage.
- **Steps:**
    1. Scroll to the sidebar section Brands.
    2. Verify that all expected brands are listed
        a. Polo,
        b. H&M,
        c. Madam,
        d. Mast & Harbour,
        e. Babyhug,
        f. Allen Solly Junior,
        g. Kookie Kids,
        h. Biba
    3. Click on each brand name and ensure the brand filter is applied
    4. Validate that the product list updates and shows only items associated with the selected brand
    5. Verify that the number of products displayed matches the count shown next to the brand name
    6. Check that brand names and product titles do not contain spelling errors

7. Collapse and expand several other brands to ensure the accordion interaction behaves consistently

Expected Results:

- All listed brands are visible and correctly labeled.
- Clicking a brand applies the correct filter and updates the product grid.
- Only products belonging to the selected brand are displayed.
- Product counts (if shown) match the actual number of displayed items.
- No spelling or formatting issues are present.

TC-04: Features Items / All Products Validation

- **Pre-conditions:** User is on homepage
- **Steps:**
    1. Navigate to the **"Products"** page from the main navigation menu
    2. Verify that product cards are listed and that each card contains the required UI elements:
        - Product image (loaded, not broken, correctly colored and sized)
        - Product name
        - Product price
        - "Add to Cart" button (enabled and clickable)
        - "View Product" button (enabled and clickable
    3. Click the **"View Product"** button and verify that the selected product page opens in the same tab
    4. Check that all product cards have consistent **width and height**, aligned properly in the grid layout.
    5. Hover over a product card and verify that the hover overlay appears according to design specifications:
        - Overlay is fully visible
        - Product name and price are readable
        - "Add to Cart" button is visible and clickable

Expected Results:

- All product cards include the required UI elements and are fully functional
- Images are correctly displayed without distortion or broken links
- Buttons are enabled and navigate to the correct pages
- Hover state matches design and behaves consistently across all cards

---

TC-05: Recommended Items Section Carousel Validation

- **Pre-conditions:** User is on homepage
- **Steps:**
    1. Scroll to the bottom of the homepage to locate the **Recommended Items** section.
    2. Verify that the **Recommended Items** section is visible
    3. Check that the carousel allows the user to move product items left and right using the navigation arrows
    4. Verify that each product card in the carousel contains the required UI elements:
        - Product image (loaded, not broken, correctly colored and sized)
        - Product name
        - Product price
        - "Add to Cart" button (enabled and clickable)

    5. Verify that **product cards do not have hover state overlays**
    6. Validate that the **left and right carousel arrows** change visually on hover and show active state when used
    7. Click through all carousel items to ensure all recommended products are displayed and functional

Expected Results:

- The Recommended Items section is visible and correctly positioned
- Carousel navigation works smoothly in both directions
- All product cards contain the required UI elements
- "Add to Cart" button functions correctly
- No hover overlays are present on product cards
- Carousel arrows respond visually on hover and active states

---

TC-06: Search Products via input field

- **Pre-conditions:** The user is on the **Products** page.
- **Steps:**
    1. Enter a valid product name or keyword into the **"Search Product"** input field
    2. Click the **Search** button or press **Enter**
    3. Confirm that only products matching the search keyword are displayed
    4. Verify that product cards contain valid details (image, name, price)
    5. Validate that unrelated products are **not** displayed
    6. Enter the same keyword in different letter cases (dress, Dress, DRESS)
    7. Verify that search results remain **consistent**, confirming that search functionality is **case-insensitive**.

- **Expected Result:**
    - The search filter is applied correctly
    - Only products matching the entered keyword appear in the results
    - Search behavior is **case-insensitive** (same results for uppercase, lowercase, or mixed input).
    - The UI updates to show the Searched Products section.

TC-07: Browse Products by Category

- **Pre-conditions:** User is on homepage
- **Steps:**
    1. Navigate to "Products"
    2. Select category Women
    3. Select subcategory Dress

- **Expected Result:** Products match category/subcategory and breadcrumb displays correct path

---

TC-08: Add Product to Cart

- **Pre-conditions:** User is on product listing page
- **Steps:**
    1. Select any product from the product listing page
    2. Click the **"Add to Cart"** button
    3. Verify that a **success notification** is displayed
        - Notification has correct styling (colors, icon, timing, position)
        - The notification text is: **"Your product has been added to cart."**
        - "**View Cart**" and "**Continue Shopping**" buttons are visible and enabled
    4. Click the **"View Cart"** button
    5. Verify the user is redirected to the **View Cart** page.
    6. On the Cart page, confirm that the product is added to the cart table with correct details:

        - Product name
        - Product price
        - Quantity (default quantity should be correct, expect to be 1)
        - Total price calculated properly (multiple items added)

- **Expected Results:**
    - Success notification appears with correct message and design

- User can navigate to the Cart page via the “View Cart” button
- The selected product is displayed in the cart with proper details and correct quantity

---

TC-09: Add Multiple Products to Cart

- **Pre-conditions:** User is on product listing page
- **Steps:**
  1. Add 3 products to cart
  2. Repeat the steps from case **TC-08: Add Product to Cart**

- **Expected Result:** All products visible, quantities and prices correct and total amount matches sum

---

TC-10: Remove Product from Cart

- **Pre-conditions:** At least one product in cart
- **Steps:**
  1. Open the **Cart** page from the top navigation menu
  2. Verify that the product table is visible and lists all items added to the cart
  3. For each product in the table, check that the following elements are displayed correctly:
     - Item name
     - Item description
     - Item price
     - Quantity
     - Total price
     - Remove (X) button

  4. Click the **“X” (Remove)** button for a selected product
  5. Verify that the product is removed from the table immediately

Expected Result: The selected product is removed from the cart table

---

TC-11: Checkout Process Validation

Pre-conditions:

- The user has at least one product in the cart
- The user is logged in to the application

Steps:

1. Open the **Cart** page from the top navigation menu
2. Verify that the product table is visible and lists all items added to the cart
3. For each product in the table, check that the following elements are displayed correctly:
    - Item name
    - Item description
    - Item price
    - Quantity
    - Total price
    - Remove (X) button
4. Verify **"Proceed to Checkout"** button is enabled and clickable
5. Click the **"Proceed to Checkout"** button
    - Button should have required design on hover and active state
6. On the Checkout page, following items should be visible:
    - address section should contain data used during registration
    - Products list with
        i. Item name
        ii. Item description
        iii. Item price
        iv. Quantity
        v. Total price
        vi. Total amount
        vii. Comment section
7. "Place order" button should be enabled
8. Click on Place order button should open payment page
9. On the Payment page, fill in the required **payment** details:
    - Name of the card
    - Card Number
    - CVC
    - Expiration
10. Click the **"Pay and Confirm order"** button
11. Verify that a success message is displayed with notification "Your order has been placed successfully!"
12. Verify "**Download Invoice**" and **"Continue"** buttons are available

13. Check that the order is recorded in the backend or user account order history.
14. Repeat checkout with invalid data to verify proper error handling.

Expected Results:

- All cart items are correctly displayed on the checkout page
- Billing details are validated, required fields cannot be left empty
- The payment method works correctly
- Order is successfully placed and confirmation message is displayed
- Invalid input triggers appropriate error messages
- Orders are correctly recorded in backend or order history

TC-12: Verify Breadcrumb Navigation

- **Pre-conditions:** User browsing subcategory
- **Steps:**
    1. Navigate to subcategory
    2. Observe breadcrumb

- **Expected Result:** Breadcrumb shows correct path; links navigate correctly.

TC-13: Contact Us Form Submission

- **Pre-conditions:** User on Contact Us page
- **Steps:**
    1. Fill all required fields with valid data
    2. Submit form
    3. Repeat with invalid data

- **Expected Result:** Valid submission shows success; invalid submission shows error

---

TC-14: Product Detail Page Validation

- **Pre-conditions:** User on products page
- **Steps:**
    1. Click on the **"View Product"** button for a selected product.
    2. Verify that the **Product Details** page opens in the same tab

3. Validate that the product details are displayed correctly, including:
    - **Product image** (loaded, not broken, correctly sized and colored
    - **Product title**
    - **Product description**
    - **Product price**
4. Verify that the **Quantity box** is enabled and allows the user to input a desired value:
    - Value can be entered manually or adjusted via increment/decrement arrows.
    - The box does **not accept invalid values** (negative numbers, letters, or special characters).
5. Verify that the **"Add to Cart"** button is enabled and clickable
6. Check the hover state of the **"Add to Cart"** button matches the design specifications
7. Validate that the **Review** section is visible and accessible

- **Expected Result:**
    1. Product details (image, title, description, price) are correct.
    2. Quantity input works as expected and restricts invalid values.
    3. "Add to Cart" button functions correctly and the hover state matches the design.
    4. The review section is present and functional.

TC-15: Product Review Submission

- **Pre-conditions:** User on Product page
- **Steps:**
    1. Fill all required fields with valid data
        - Name, email and review comment
    2. Click on "Submit" button
    3. Verify that a **success notification** appears with the message: "Thank you for your review."
    4. Verify that the notification disappears automatically after a few seconds.
    5. **NICE TO HAVE**: Validate that the review is stored correctly in the **database** or the required external review management tool
    6. Repeat the test with **invalid data** (e.g., empty fields, invalid email) to verify that proper error messages are shown.

- **Expected Result:**

    1. Valid review submission displays a success notification
    2. Invalid submissions trigger appropriate error messages
    3. Reviews are correctly stored in the backend or external tool
    4. Notification behavior (appearance and disappearance) works as intended

---

Bug reports

The purpose of the Bug Reporting section is to define a clear and standardized process for identifying, documenting, and tracking defects discovered during testing. All issues found during test execution must be reported in a consistent format to ensure effective communication between QA, development, and product teams.

Bug reports should include detailed reproduction steps, expected and actual results, environment information, severity and priority classification, as well as any relevant attachments (screenshots, logs, videos). This ensures that defects can be reproduced reliably, prioritized accurately, and resolved efficiently.

BUG-01: Wrong Products Assigned to Wrong Category

Title: Products appear in incorrect category ("Dress" category shows Shirt/T-Shirt items)
Environment: Production – https://automationexercise.com
Severity: Medium
Priority: High

Pre-conditions: User on “Product” page.

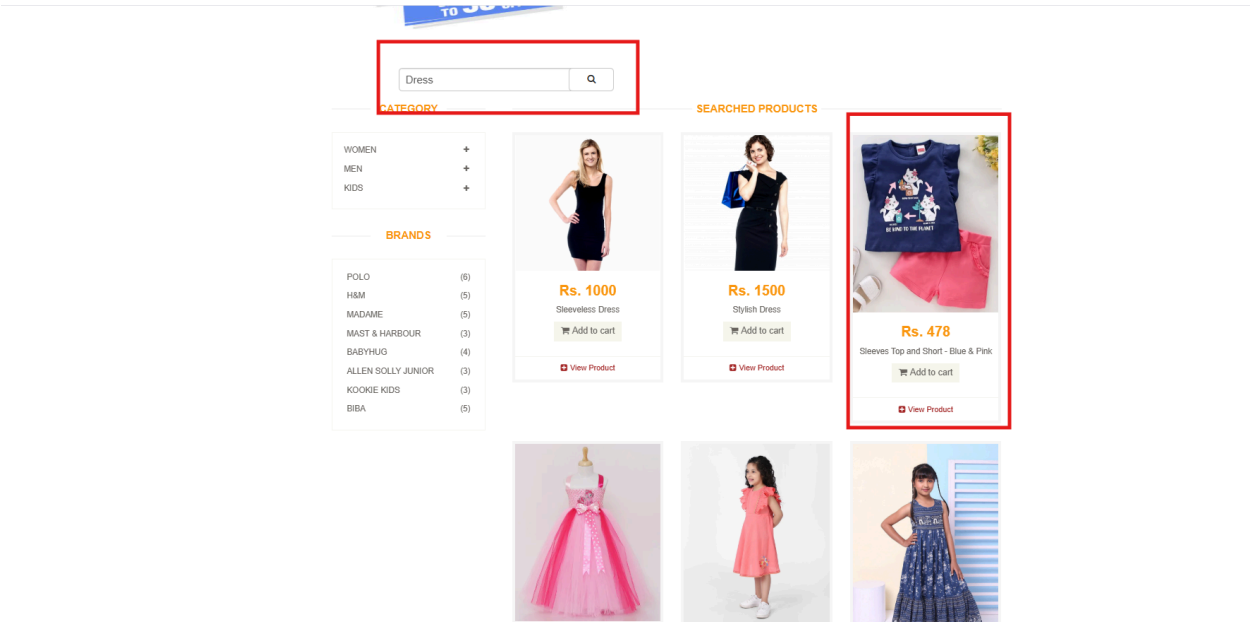
Steps to Reproduce:

1. In ‘Search Product” input field put “Dress”
2. Click on Enter or Search button
3. Check the product lists
4. See: https://automationexercise.com/category_products/4

Expected Result: Only products belonging to the **Dress** category should be displayed

Actual Result: Products such as **T-Shirt** and **Shorts** appear in the Dress category

Attachment:



BUG-02: Missing 404 Page for Invalid Product Link

Title: Invalid product URL does not return a proper 404 page
Environment: Production – https://automationexercise.com
Severity: Medium
Priority: Medium

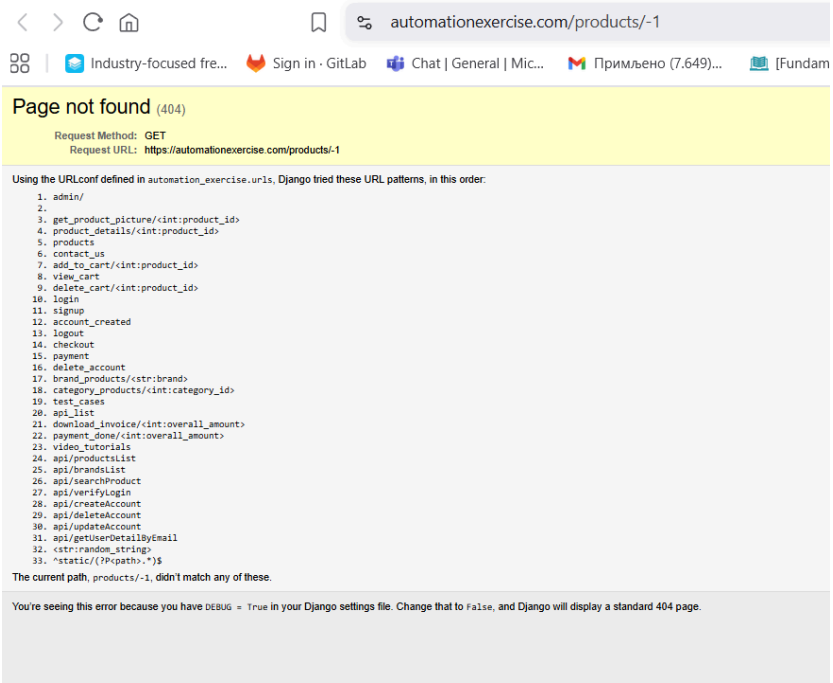
Steps to Reproduce:

1. Open an invalid product link, e.g.: https://automationexercise.com/products/-1

Expected Result:
User is redirected to a proper **404 Not Found** page or given a meaningful error message

Actual Result:
The page loads with inconsistent or broken content, without any 404 handling.

Attachment:



BUG-03: Misspelled Category Name ("Tshirts")

Title: Misspelling in Men → T-shirts category
Environment: Production – https://automationexercise.com
Severity: Low
Priority: Low

Pre-conditions: User is on Home page

Steps to Reproduce:

1. Click on accordion button in **Men** category section
2. When the section is expanded, check the subcategory label “Tshirts”.

Expected Result: Correct spelling: **T-shirts**

Actual Result: Displayed as: **Tshirts** (missing hyphen)

BUG-04: Image Rendering / CSS Issue on Product Page

Title: Product images not properly styled on product page
Environment: Desktop – https://automationexercise.com
Severity: Medium
Priority: Medium

Pre-conditions: User on Product page

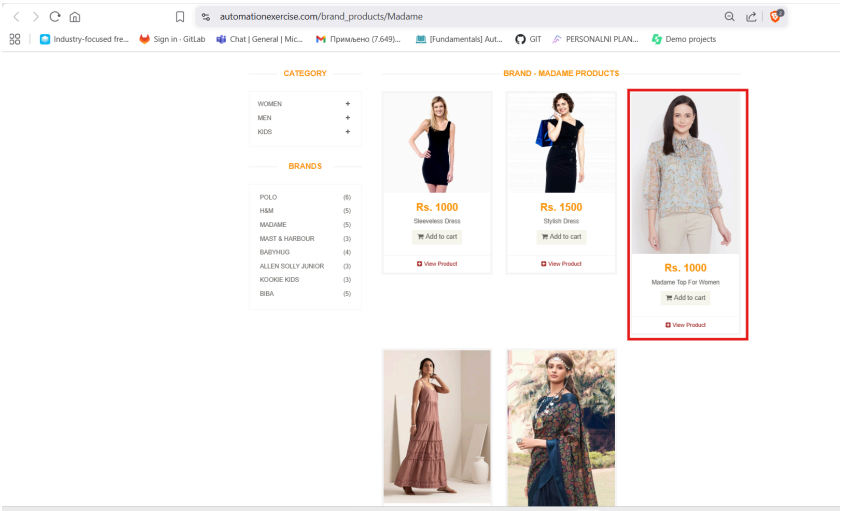
Steps to Reproduce:

1. Navigate to: https://automationexercise.com/brand_products/Madame
2. Observe product images

Expected Result:
All images should have the same width and height

Actual Result:
Images appear stretched, poorly aligned or incorrectly resized

Attachment:



BUG-05: No Limitation on Adding Same Product to Cart

Title: System allows adding invalid or excessively large quantities of the same product to the cart
Environment: Production – https://automationexercise.com
Severity: High
Priority: High

Pre-conditions: The user is on the product details page or product list page.

Steps to Reproduce:

1. Navigate to https://automationexercise.com/product_details/16
2. In the Quantity input field, enter an invalid or extremely large value (e.g., **10000** or **-1**).
3. Click the **"Add to Cart"** button.

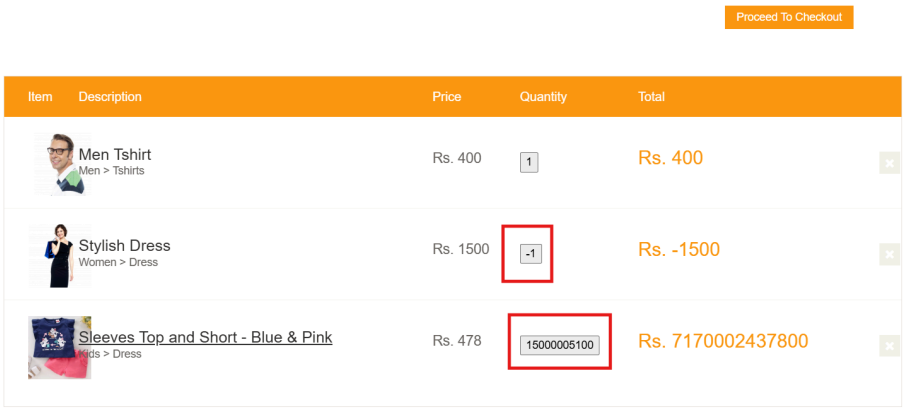
Expected Result: The system should validate quantity input and restrict:

- negative numbers
- zero quantity
- excessively large numbers
- non-numeric input

It should display a proper validation error or set a maximum allowed value.

Actual Result:
The system accepts invalid and extremely large quantity values and adds them to the cart without any validation.

Attachment:



BUG-06: Breadcrumb Navigation Not Working

Title: Breadcrumb navigation fails to follow correct navigation path
Environment: Production – https://automationexercise.com
Severity: Medium
Priority: High

Pre-conditions: User on product page

Steps to Reproduce:

1. Navigate to the product page
2. Click on any category
3. Select any subcategory

4. Observe breadcrumbs
5. Try to click on breadcrumb links

Expected Result:
Breadcrumb shows the correct navigation path and links redirect to the appropriate pages, allowing user to easily navigate / back to the previous page

Actual Result:
Breadcrumb links do not lead to expected pages